[41] Keyulu Xu, Jingling Li, Mozhi Zhang, Simon S Du, Ken-ichi Kawarabayashi, and Stefanie Jegelka. What can neural networks reason about? *arXiv preprint arXiv:1905.13211*, 2019.

[42] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik R Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. URL https://openreview.net/forum?id=5Xc1ecxO1h.

[43] Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of language models: Part 2.1, grade-school math and the hidden reasoning process, 2024. URL https://arxiv.org/abs/2407.20311.

[44] Yizhou Zhang, Lun Du, Defu Cao, Qiang Fu, and Yan Liu. An examination on the effectiveness of divide-and-conquer prompting in large language models, 2024. URL https://arxiv.org/abs/2402.05359.

[45] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V Le, and Ed H. Chi. Least-to-most prompting enables complex reasoning in large language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WZH7099tgfM.

Appendix

Contents

A Limitations

This proof-of-concept study highlights the critical role of aligning Chain-of-Thought (CoT) length with model capability and task difficulty, particularly in training data. However, accurately estimating the optimal CoT length in real-world scenarios remains challenging due to the complexity and variability of reasoning tasks. While our theoretical and empirical analyses offer practical heuristics for coarse approximation, they may not fully capture the nuances of diverse problem domains or model behaviors. We leave the development of more precise estimation methods and adaptive strategies for optimal CoT length selection in complex, real-world settings as promising directions for future work.

B Formal Definitions of Simplified Arithmetic Problem

To begin, we aim to empirically investigate the relationship between reasoning performance and CoT length. Therefore, we need to control a given model to generate reasoning chains of varying lengths for a specific task. Unfortunately, no existing real-world dataset or model fully meets these strict requirements. Real-world reasoning tasks, such as GSM8K or MATH [9, 18], do not provide multiple solution paths of different lengths, and manually constructing such variations is challenging. Moreover, it is difficult to enforce a real-world model to generate a diverse range of reasoning paths for a given question. Given these limitations, we begin our study with experiments on synthetic datasets.

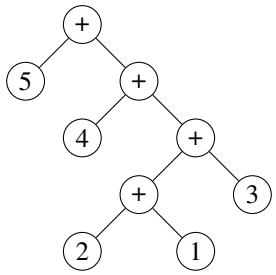

Figure 5: Computation tree of arithmetic expression $5 + (4 + ((2 + 1) + 3))$.

B.1 Problem Formulation

To investigate the effect of CoT length in a controlled manner, we design a synthetic dataset of simplified arithmetic tasks with varying numbers of reasoning steps in the CoT solutions.

Definition B.1 (Problem)**.** In a simplified setting, an arithmetic task q is defined as a binary tree of depth T. The root and all non-leaf nodes are labeled with the $+$ operator, while each leaf node contains a numerical value (mod 10). In addition, we impose a constraint that every non-leaf node must have at least one numerical leaf as a child.

The bidirectional conversion method between arithmetic expressions and computation trees is as follows: *keeping the left-to-right order of numbers unchanged, the computation order of each "+" or tree node is represented by tree structure or bracket structures.* For example, consider the task $5 + (4 + ((2 + 1) + 3))$ with $T = 4$. The corresponding computation tree is defined as Figure 5.

To ensure that CoT solutions of the same length have equal difficulty for a specific problem, we assume that each reasoning step performs the same operations within a single CoT process.

Definition B.2 (Solution)**.** We define a t-hop CoT with a fixed each step length of t as a process that executes t operations starting from the deepest level and moving upward recursively.

According to this definition, the execution sequence is uniquely determined. For example, one way to solve expression in Figure 5 is by performing one addition at a time:

$$5 + (4 + ((2 + 1) + 3)) = \texttt{<1>} \tag{3}$$
$$2 + 1 = 3 \tag{4}$$
$$3 + 3 = 6$$
$$4 + 6 = 0$$
$$5 + 0 = 5\texttt{<END>.}$$

Another approach is to perform two additions at a time:

$$5 + (4 + ((2 + 1) + 3)) = \texttt{<2>} \tag{5}$$
$$(2 + 1) + 3 = 6$$
$$5 + (4 + 6) = 5\texttt{<END>.}$$

The latter approach is half as long as the former, but each reasoning step is more complex[6]. This illustrates a clear trade-off between the difficulty of each subtask and the total number of reasoning steps.

[6]This is because performing two operations at once requires the model to either memorize all combinations of numbers in a two-operator equation and their answers, apply techniques like commutativity to reduce memory requirements, or use its mental reasoning abilities to perform the two operations without relying on CoT.

In practice, when t does not evenly divide T, the final step performs $T \mod t$ operations. To guide the model in generating the desired CoT length, we insert the control token `<t>` after the question and before the beginning of the solution. To preserve the parentheses that indicate the order of operations, we construct expressions in Polish notation. However, for readability, we present each problem in its conventional form throughout the article.

B.2 Contrast to vanilla arithmetic problem

Why pruning? Initially, we intended to create a synthetic dataset for regular arithmetic tasks, but we quickly realized that the computation tree for such tasks is uncontrollable. For example, consider the task $1 * 2 + 3 * 4$. We hoped to compute 2 operators in one step, but found it impossible because the addition needs to be computed after the two multiplications, and we cannot aggregate two multiplications in one subtask. Therefore, pruning the computation tree becomes essential.

Why only focusing on addition? There are two reasons why we focus on arithmetic tasks involving only addition: first, it simplifies pruning, as the order of operations can be controlled solely by parentheses; second, it facilitates the computation of sub-tasks, since parentheses do not affect the final result, and the model only needs to compute the sum of all the numbers when solving a sub-task. We aim for the model to handle longer sub-tasks, thereby allowing a broader study of the impact of CoT length.

Will the simplified synthetic dataset impact the diversity of the data? We need to clarify that even with pruning, the structure of the expressions will still vary because swapping the left and right child nodes of each non-leaf node in the computation tree results in different expressions. When $T > 30$, the number of possible variations exceeds 1×10^9.

C Supplementary Details on Real world Experiment for Optimal CoT Length

C.1 Implementation Details

Solution Length Control. To study the impact of CoT length on performance under a given problem difficulty, we need to induce the model to naturally generate solutions of varying lengths. Simply adding prompts like "*please use 100 tokens to solve this problem*" or "*please use 10 steps to solve this problem*" is not ideal because the model's ability to follow instructions regarding output length is limited, and such fixed-length prompts may not ensure fairness across problems of different difficulties. Moreover, prompting for a specific length might lead the model to generate irrelevant tokens or steps just to "pad the length," without actually changing the number of steps or the complexity of the reasoning. Additionally, controlling `max_length` is also problematic, as overly long responses might get truncated, which would directly lead to lower accuracy for longer outputs. What we really want is for the model to generate a complete and coherent long response on its own, so we can observe the corresponding accuracy.

To create solutions with varying step lengths with different complexity, we follow [12] by using in-context examples (8-shots) with three different levels of complexity to guide the model in generating solutions with different step counts. For each set of in-context examples, we sample 20 times, resulting in a total of 60 samples per question.

Step Segmentation. Simply measuring CoT length by counting tokens is neither rigorous nor meaningful. Since our focus is on final performance rather than efficiency, we care more about using CoT length to reflect the complexity of the reasoning pattern. In this sense, the number of reasoning steps can serve as a more appropriate indicator of CoT length. As we discussed earlier, the step number captures how the model decomposes the problem, which directly reflects the complexity of its reasoning. In contrast, token length fails to capture this because, as the model thinks more deeply and the number of steps increases, the number of tokens per step may decrease—making the total token count unpredictable and unreliable as a proxy for reasoning complexity.

When calculating the number of steps, we separate the full reasoning chain using `"\n"`[12] and remove empty lines caused by `"\n\n"`. Then we consider the total number of lines as the CoT length. Since questions in the MATH dataset are challenging and lead to high variability in final CoT lengths, we normalize the lengths by applying `length = length // bin_width`. For experiments comparing different models (e.g., optimal CoT length per model or optimal vs. longest CoT), the